

FIT2109 Computer Science Workshop

Week 4: Version Control with Git — Foundations

Zahra Mirzamomen

Department of Software Systems and Cybersecurity

Faculty of IT, Monash University

Learning outcomes

By the end of this week you should be able to

- Explain why version control exists, and Git's snapshot model.
- Set up Git and describe its three areas.
- Record changes in deliberate, inspectable steps.
- Keep build output, dependencies, and secrets out of history.
- Write clean history, and fix your latest mistake without clutter.
- Understand Branching, merging, conflict resolution, and stashing unfinished work.

How today works

The same rhythm, four times

For each activity: a quick **recap** of the concepts you need, a few short **emos** to watch, then you **do the activity** in your group of 3–4.

Activity	Time
1 — Create a repo & why Git is useful	10 min
2 — Repository model and git status	20 min
3 — Commits, DAGs and reading history	20 min
4 — Clean history, branching & conflicts	25 min

What is a version control system?

Definition

A system for recording changes to files over time — so you can recall earlier versions, compare revisions, see exactly what changed, and see who changed it.

Without one

You end up relying on duplicate folders and renamed copies like `assignment_v2_FINAL.doc` — with no clear history, no easy way to review changes, and no disciplined way to recover from a mistake.

The scenario it solves

You rewrite the login code across five files. Two hours in, nothing works — and you can't tell whether it's the rewrite or some other edit along the way. You want to compare against *exactly* how it looked this morning, the last point where everything worked. But you never marked that moment.

Introducing Git

A popular version control system

Git is a free, open-source, **distributed** version control system created by Linus Torvalds in 2005 — the one almost the entire software industry has settled on.

Repository (repo)

A project Git is tracking, plus its entire history — and every clone holds all of it, not just the latest state.

Snapshots, not diffs

Each commit is a **picture of every file** at that moment, not a list of differences. Unchanged files are reused rather than stored again.

Three things Git provides

History

Inspect how the project evolved.

Recovery

Compare & revert instead of guessing.

Coordination

Many people, one detailed record.

Recap: history

Commit

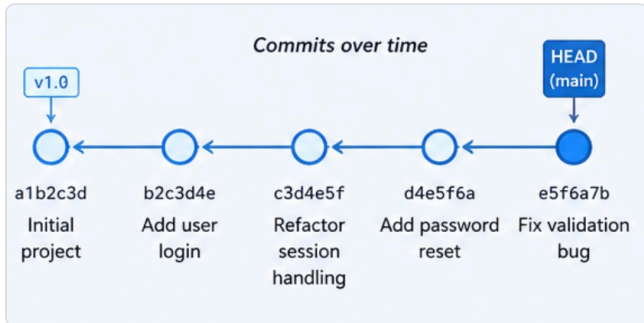
A saved snapshot of the project at one point in time.

HEAD & main

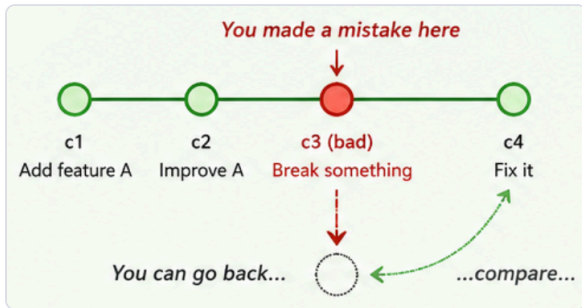
`HEAD` is the commit you have checked out right now; `main` is the branch currently pointing at it.

v1.0

A tag — a marker that permanently pins one exact commit, such as a release.



Recap: recovery



When a commit breaks something, you can go back to before it happened and compare — instead of guessing.

Find when it changed

`git log` walks history commit by commit, so "when did this still work?" has a real answer.

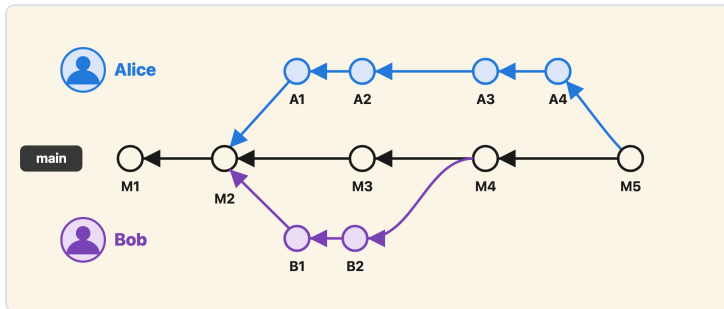
See exactly what changed

`git diff` shows the precise lines added and removed — not a vague "something broke".

Go back

`git restore` returns a file to its last committed state, structurally rather than by guesswork.

Recap: coordination



Two people can work on separate lines of development that later rejoin — each merge is its own commit with exactly two parents.

Not just for teams

Even working alone, this is what lets you break work into meaningful, reviewable steps instead of one undifferentiated blob of files.

What the record answers

Who changed what, when, and why — and months later, exactly which change introduced a bug.

Recap: git config

Set once per machine, before your first commit

```
git config --global user.name "Ada Lovelace"  
git config --global user.email "ada@example.com"  
git config --global init.defaultBranch main
```

Why

Every commit is permanently stamped with this name and email — it shows up in every log, every blame, forever.

Windows users: line endings

Git for Windows defaults to converting files to CRLF on checkout, which can break shell scripts on Linux/WSL. Set `git config --global core.autocrlf input` instead — commits stay LF, checkouts are left untouched.

Demo: watch — creating a repository

Watch first

We'll set up a repository together and take a first look at its status.

Commands covered

```
git init  
git status
```

What to notice

`git init` creates a hidden `.git` folder — that *is* the repository. `git status` reports "No commits yet": nothing is tracked until you `git add` it.

Activity 1

10 min

Now

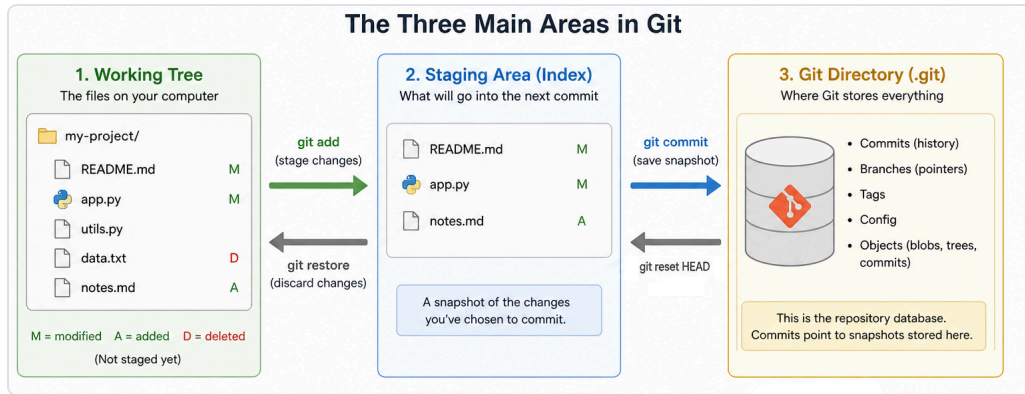
Get the workshop handout from **Ed Lessons** and complete Activity 1 with your group.

Discuss with your group

Work through the activity's questions and challenges together.

Recap: the three areas, illustrated

The Three Main Areas in Git



Working Tree

The actual files on disk right now.

Staging Area

What will go into the next commit.

.git Directory

Permanent history & metadata.

git status answers three questions

Three states

- **Modified** — changed in the working tree, not yet staged.
- **Staged** — marked to go into the next commit.
- **Committed** — safely recorded in the local database.

What it answers

What's already staged? What's changed but unstaged? What's untracked?

Example: Output of `git status`

```
$ git status
```

Shows the state of your working tree and staging area.

```
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
        new file:   app.py
        new file:   utils.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py
        modified:   utils.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data/
```

Branch Information

You are on branch `main`, which is in sync with `origin/main`.

Staging area

Files listed here are staged and will go into the next commit.

Working tree (modified)

Files changed in your working directory but not yet staged.

Untracked files

Files in your working directory that Git is not yet tracking.

Recap: the core command cycle

Command	What it does
<code>git init / git clone <url></code>	Start a repo locally, or copy one (full history) from elsewhere.
<code>git status</code>	Staged / modified / untracked, right now.
<code>git add <file></code>	Stage a snapshot of this file's current content.
<code>git diff / --staged</code>	What's unstaged? / What will be committed?
<code>git commit -m "..."</code>	Record exactly the staged snapshot.
<code>git log</code>	Walk the history via parent links.

Start a brand-new project

```
git init
git status  # app.py is untracked
git add app.py
git commit -m "Add initial app script"
```

Join a project that already exists

```
git clone <url>
cd assignment
git status  # clean, in sync
git log --oneline  # their history
```

Demo: watch — stage, inspect, commit

Watch first

We'll edit a file, stage it, inspect it, and commit it.

Commands covered

```
git status  
git add  
git diff  
git commit  
git log
```

What to notice

The file moves between sections of `git status` — nothing is ever silently lost by staging or committing.

Recap: .gitignore

Purpose

Keeps build output, dependencies, editor settings and secrets out of `git status` and `git add .`

Pattern syntax

- / — a whole directory.
- * — wildcard: *.pyc by extension.
- ! — re-include what a broader rule excluded.

A typical .gitignore

```
# Python
__pycache__/
*.pyc
# Node + editors
node_modules/
.DS_Store
```

Gotcha: it only affects files Git isn't already tracking

```
git rm --cached secrets.env # untrack, keep on disk
echo "secrets.env" >> .gitignore
git add .gitignore && git commit -m "Stop tracking secrets.env"
```

Commit `.gitignore` itself — and note that untracking a secret now does *not* erase it from earlier commits.

Recap: undoing a stage, fixing a commit

```
git restore <file>
```

Discards uncommitted edits — back to the last commit. This cannot be undone.

```
git restore --staged <file>
```

Unstages the file — moves it back to modified. Your edits are kept, just not queued.

Reading the hint text

`git status` prints the exact command to run under each section — that's not decoration, it's the fix.

Hint: `git add` stages content, not files

Every `git add <file>` snapshots that file's *current* content — not a one-time "start tracking" switch. Edit again, and you must `add` again.

Forgot a file? Amend the tip commit — only if unshared

```
git add tests/test_validation.py
git commit --amend --no-edit # keep the message
git commit --amend -m "Better message" # or rewrite it
```

Demo: undoing a stage or commit

Watch first

We'll edit a file, stage it, commit it, then undo the stage or the commit.

Commands covered

```
git restore  
git reset
```

What to notice

`git restore` fixes a file; `git reset` moves where `HEAD` points — use `restore` for uncommitted changes, `reset` to undo a commit.

Recap: clean commit habits

1. Logical grouping

One commit, one idea — stage unrelated changes separately.

2. Inspect before committing

`git diff --staged` catches stray lines before they're permanent.

3. Meaningful messages

Imperative title, blank line, then detail if needed.

4. Don't blindly `git commit -a`

Stages every tracked change, including ones you forgot.

5. Amend, don't stack

`git commit --amend` fixes the tip commit — only if unshared.

6. Meaningful checkpoints

Not one giant commit, not noisy micro-commits.

7. Keep every commit working

A broken commit can't be tested, and breaks `git bisect` later.

8. Never commit secrets

Deleting the file later doesn't remove it from history.

Now

Get the workshop handout from **Ed Lessons** and complete Activity 2 with your group.

Discuss with your group

Work through the activity's questions and challenges together.

Recap: branches & the DAG

What is a branch?

A movable pointer to a commit — not a copy of the project.

Use cases

Isolate a feature, experiment safely, or work in parallel without disrupting `main`.

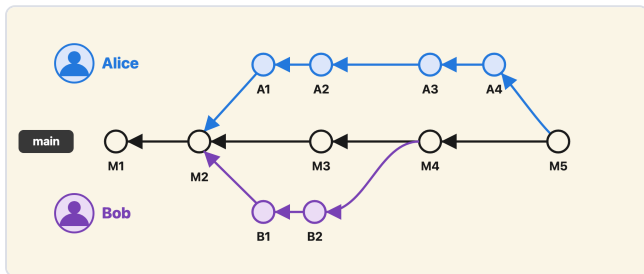
Parent count and the DAG

Initial commit → **0** parents.

Normal commit → **1** parent.

Merge commit → **2+** parents.

Never loops — that's a **directed acyclic graph**.



Recap: branch, switch & merge commands

Command	What it does
<code>git branch</code>	Lists local branches.
<code>git branch -d <name></code>	Deletes a branch — only if fully merged. Safe default.
<code>git branch -D <name></code>	Force-deletes even if not merged. Use deliberately.
<code>git switch <branch></code>	Switches to an existing branch.
<code>git switch -c <new></code>	Creates a new branch and switches to it.
<code>git switch --detach <tag></code>	Checks out a commit or tag without a branch.
<code>git merge <branch></code>	Merges the named branch into the current one.

The everyday feature-branch workflow

```
git switch -c feature-readme # branch off, move onto it
# ... edit, git add, git commit ...
git switch main # back to the main line
git merge feature-readme # then git branch -d it
```

Recap: git log

What it does

Walks history via parent links — hash, author, date, message for every reachable commit.

Common options

- `--oneline` — short hash + message.
- `--graph --decorate --all` — graph, all branches.
- `-N` — N most recent commits.
- `--author="name"` — filter by author.
- `A..B` — reachable from B, not A.

git log --oneline --graph --decorate --all

```
* 8d4c7a1 (HEAD -> main) Merge 'feature-notes'
| \
| * 6e2b9f4 (feature-notes) Add release notes
* | 1c8f5a3 Update README
| /
* 4b7e2d8 Add shared setup
* 9f3a1c2 Initial commit
```

* marks a commit; | \ / are just connecting lines. The graph splits, then rejoins at the merge commit.

Demo: watch — create a branch and inspect with git log

Watch first

We'll branch, commit on both sides, merge, then read the graph together.

Commands covered

```
git branch  
git switch  
git merge  
git log --graph
```

What to notice

The graph shows the split *and* the rejoin — a plain chronological list could never show that.

Recap: git stash

Use case

Not ready to commit, but need to switch branches now? `git switch` refuses if it would overwrite your changes.

Commands

- `git stash` — shelve changes, restore a clean tree.
- `git stash pop / apply` — bring them back (pop removes, apply keeps).
- `git stash list / drop` — see or discard stashes.
- `git stash -u` — also shelve untracked files.

The workflow: an urgent bug arrives mid-task

```
git stash # shelve WIP, tree goes clean
git switch main # fix the urgent issue, commit it
git switch feature-readme
git stash pop # your WIP is back
```

A stash is local and disposable — never pushed, never part of commit history.

Demo: watch — using git stash

Watch first

We'll stash and restore work in progress before switching between branches.

Commands covered

```
git stash  
git stash list  
git stash pop
```

What to notice

Stashing never deletes anything — your changes sit safely in the stash list until you pop or drop them.

Now

Get the workshop handout from **Ed Lessons** and complete Activity 3 with your group.

Discuss with your group

Work through the activity's questions and challenges together.

Recap: fast-forward vs true merge

Fast-forward

No divergence — Git just moves the branch pointer forward. No new commit.

True merge

Branches diverged — Git creates a new merge commit with **two parents**.

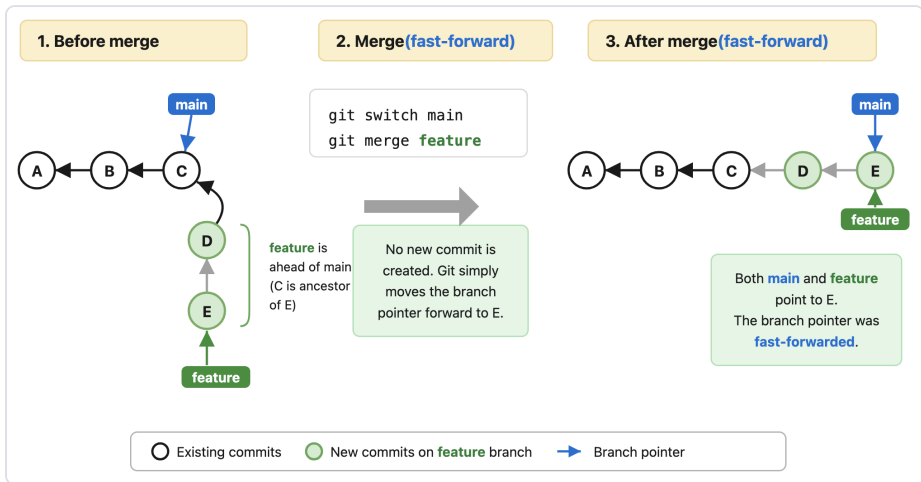
How Git decides — you don't choose

If your current branch tip is an **ancestor** of the branch you're merging in, nothing has diverged, so Git just slides the pointer forward. If both branches have moved on since they split, Git must build a merge commit to record that two lines of work came back together.

Why branching is cheap

A branch is a lightweight movable pointer, not a copy of the repository — `git switch -c <name>` is instant.

Recap: fast-forward merge, illustrated



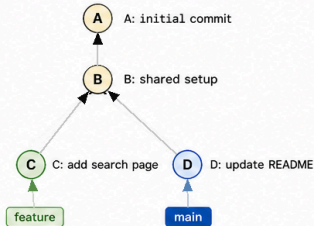
No new commit is created — Git simply moves the `main` pointer forward to match `feature`.

Recap: true merge, illustrated

1) Setup: create commits that diverge

- 1 `git init -b main`
- 2 `echo "Initial app" > app.txt`
`git add app.txt`
`git commit -m "A: initial commit"` A
- 3 `echo "Shared setup" >> app.txt`
`git commit -am "B: shared setup"` B
- 4 `git switch -c feature`
- 5 `echo "Search page" > search.txt`
`git add search.txt`
`git commit -m "C: add search page"` C
- 6 `git switch main`
- 7 `echo "Project notes" > README.md`
`git add README.md`
`git commit -m "D: update README"` D

2) Before merge (branches have diverged)

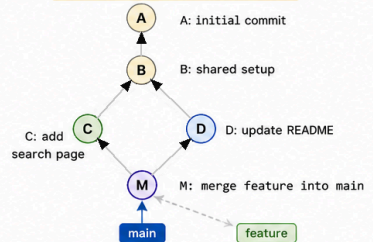


`main` has commit `D`.
`feature` has commit `C`.
Neither is an ancestor of the other.
Therefore, Git cannot fast-forward.

3) Merge `feature` into `main`

```
git switch main  
git merge feature -m "M: merge feature into main"
```

4) After merge (true merge)



Recap: merge conflicts

Why it happens

The same lines of a file were changed differently on both branches.
Git can't guess which version is correct, so it stops the merge and asks you to decide.

What you'll see inside the file

```
<<<<<< HEAD
APP_TITLE = "Log Tool"
=====
APP_TITLE = "Log Inspector"
>>>>>> feature-title
```

Resolving it

`git status` lists the unmerged file → edit out the markers to the version you want → `git add` → `git commit`. Not sure yet? `git merge --abort` backs out cleanly.

Demo: watch — branch, merge & resolve a conflict

Watch first

We'll create a conflict on purpose, then resolve it.

Commands covered

```
git switch  
git merge  
git add  
git commit
```

What to notice

Git only stops when the *same lines* changed on both sides — it merges everything else automatically.

Now

Get the workshop handout from **Ed Lessons** and complete Activity 4 with your group.

Discuss with your group

Work through the activity's questions and challenges together.

Coming up: Week 5

Today was entirely local

One person, one machine. Week 5 adds a second person and a shared remote.

What's next

- `fetch / pull / push` and remote-tracking branches
- Pull requests & collaborative workflows
- Rebase, interactive rebase & safe force-pushing
- `git bisect`, `revert`, and `reset`

Exit ticket

Before next time

Practice the Git commands we learned this week and repeat the workflows a few times until you're comfortable with it..

Exit ticket

Wrap up with the PollEv activity!